HyQEC: constructing new quantum LDPC codes and certifying their distances

Minbo Gao
Hyra Team, Tencent Hunyuan

Technical report, 3 November 2026.

Abstract

Quantum computers need error-correcting codes, which combine many fragile physical qubits into fewer, more reliable logical qubits, and the cost of this protection is a central obstacle to building them. Promising low-overhead codes, notably quantum low-density parity-check (qLDPC) codes, are being found at a fast pace, but their most important property, the distance, which measures how many errors a code can withstand, is hard to prove: reported distances are often estimates, and on a public leaderboard of one class of such codes every distance was an estimate. We present HyQEC, a system that constructs new codes, searches for their weak points and proves their distances, leaving proofs that a separately written program can check. It was developed by the AI agent system Hyra 2.0 under human direction. HyQEC finds a code that stores 123 logical qubits in 488 physical qubits at distance 22 and scores 48% higher on the standard figure of merit than the previous leader of its category, with both distances proven exactly; further new codes score higher still or need even fewer physical qubits per logical qubit. Applied to the leaderboard, HyQEC shows three claimed distances to be too high, including that of the top entry, and establishes 178 of the 187 claimed distances exactly: 125 by both branch and bound and satisfiability (SAT) checks, and 53 by branch and bound alone. It outperforms the compared tools on selected large, structured codes, while existing tools remain faster on many small ones.

1 Introduction

Quantum computers promise to solve some problems that are out of reach for ordinary computers, but their basic building blocks, physical qubits, are fragile: they pick up errors far more often than the transistors of a conventional chip. The standard remedy is quantum error correction. Many physical qubits are combined into a code that stores a smaller number of more reliable logical qubits, and errors are detected and fixed while the computation runs. A code is described by how many physical qubits it uses, how many logical qubits it stores, and its distance: the smallest number of errors that can corrupt the stored information without being noticed. The larger the distance, the more errors the code can correct.

This protection is expensive. In the most widely studied scheme, the surface code, a logical qubit can require hundreds of physical qubits[1,2], and even after recent improvements, estimates for one landmark algorithm, factoring a 2048-bit number, call for fewer than a million physical qubits[3,4]. A family known as quantum low-density parity-check (qLDPC) codes can cut this cost substantially[5–8]: in one well-known example, 288 physical qubits, including measurement ancillas, protect 12 logical qubits about as well as nearly 3,000 would with the surface code under the studied noise conditions[9]. Finding good codes of this kind, at sizes that hardware could realistically build, has therefore become an active search. A public leaderboard run by the Unitary Foundation collects the best codes found and ranks them by a standard figure of merit, which rewards storing many logical qubits at a large distance with few physical qubits[10]. It keeps a separate board for a less-studied type of code whose checks mix different kinds of measurement, which gives more freedom of design but makes the codes harder to analyse.

The weak point of this search is the distance. It is the number that matters most, because the figure of merit grows with its square, and it is also the hardest to establish. Finding a single harmful error pattern shows that the distance is at most the size of that pattern, and computer searches can do this quickly. Proving that no smaller harmful pattern exists, however, means ruling out an astronomically large number of candidates, more than 10^{46} for one of the codes in this report. Such proofs run for hours or days on large computers, and a single programming error can make a wrong answer look right. Many reported distances are therefore estimates rather than proven values; on the leaderboard's board for mixed-check codes, every distance was an estimate when this work began.

We present HyQEC, a system that discovers new codes and proves their distances, so that every result it reports is either proven or clearly marked as an estimate. Its proofs also leave records that a separately written program can check, so that others need not take a long computation on trust. HyQEC makes three contributions.

- *New codes* (Section 3). HyQEC builds codes in three different ways. Its headline code uses 488 physical qubits to store 123 logical qubits at distance 22; its figure of merit is 48% higher than that of the previous leader of its category, and both distances are proven exactly. Another new code scores higher still, with a proof that has so far been run once. A higher figure of merit means better code parameters; whether it lowers the cost of a working fault-tolerant machine also depends on decoding and hardware, which this report does not evaluate.
- *Proofs that others can check* (Sections 2 and 5). A fast search on graphics processors finds harmful error patterns, which bound the distance from above. Two independent exhaustive methods prove that no smaller pattern exists, and their proofs are recorded in certificates that a separate checker can re-run.

- *A check of public claims* (Section 4). Applied to the 187 codes on the leaderboard's board for mixed-check codes, HyQEC found three claimed distances to be too high, including that of the top entry, and proved the claimed distance exact for 178 of them.

HyQEC was developed by Hyra 2.0, a system of AI agents, working to goals and standards set by a human researcher (Section 6). Section 2 introduces the terms and explains how HyQEC establishes a distance; Sections 3 and 4 present the new codes and the audit of the leaderboard; Section 5 explains why the proofs can be trusted and where they stop; and Section 6 discusses what the results do and do not establish. Algorithms, proofs, full benchmarks, defect records and the development record are in the Supplementary Information (SI).

2 HyQEC in brief

Terms. The report uses a few terms throughout; formal definitions are in SI Section S1.

- $[[n,k,d]]$ denotes a code that uses n physical qubits to store k logical qubits and has distance d.
- A code is defined by its *checks*, measurements of products of the single-qubit Pauli operators X, Y and Z that reveal errors without disturbing the stored information. The *check weight* w is the largest number of qubits on which a check acts.
- An undetectable error that changes the stored information is a nontrivial *logical operator*, and d is the smallest number of qubits such an error can act on. A logical of weight u, a *witness*, proves the upper bound $d \leq u$; excluding every lighter logical proves a lower bound, and the distance is *exact* when the two bounds meet.
- In a *CSS* code every check is a product of X operators only or of Z operators only; in a *non-CSS* code checks may mix Pauli types. The leaderboard's board for such codes is its non-CSS board.
- Codes are compared by the figure of merit kd^2/n, which rewards rate and distance together, and by the overhead n/k, the number of physical qubits per logical qubit.

What was missing. Computing the distance is NP-hard[11,12]. Randomized information-set searches[13–15] give upper bounds, and exhaustive methods[16–18] give lower bounds at a cost that grows combinatorially with the code. Before this work, every distance on the non-CSS board was recorded only as an upper bound[10]. Exact distances of large codes had been reported with search records as their only evidence, up to distance 20 for the non-CSS codes in our computational survey[17], while distances checked by SAT certificates or proof assistants covered mainly CSS codes, the only non-CSS example we found being the five-qubit code $[[5,1,3]]$[19,20]. AI systems that search for qLDPC codes report heuristic distances[21,22], and He, Lu and Zeng report a Lean-verified catalogue of 14,116 nonadditive quantum codes in their revised preprint[23]. SI Table S7 collects the reported reach of exact methods.

How a distance is established. HyQEC separates finding a weakness from proving that no smaller weakness exists (Fig. 1). A search on graphics processors looks for a logical of some weight u; once verified, it proves $d \leq u$. An exhaustive branch-and-bound search then rules out every lighter logical, which proves $d \geq u$; its pruning rules are proved never to discard a logical within the search bound (SI Theorem S1). The search is cut into tasks that can be replayed one by one, so it runs on CPUs and on GPUs with identical node counts, and its record can be re-executed by a separately written checker. A second method, which encodes the question as a satisfiability problem and has every refutation checked by the standard tool drat-trim[24], gives independent evidence where it is feasible. For $[[488,123,22]]$, a verified weight-22 logical gives $d \leq 22$, and an exhaustive search of 3.16×10^{12} nodes, run on CPUs and GPUs with identical totals and re-executed by the checker, excludes every weight up to 21, so $d = 22$ exactly.

How strong the evidence is. Not every distance rests on the same evidence. Upper bounds always come from verified witnesses. For lower bounds, Table 1 lists the kinds of evidence used and the results that rest on each; we state the kind for every result rather than a single label such as "certified". Values taken from the board or the literature are marked as reported.

3 New codes

The board's leader among weight-8 non-CSS codes, $[[452,115,18]]$ with $kd^2/n = 82.4$, is a reversing fold of its 904-qubit circulant CSS parent[10]. It uses symplectic halving[17] with a pair-partition parent[25]. Folding pairs up the qubits of a larger parent code and merges each pair into one, halving the code. It keeps the parent's strength only if no light logical of the parent covers both qubits of many pairs, because a pair that is covered fully counts once instead of twice after merging. For the ten weight-8 parents tested, verified logicals prove a distance of at most 18 for all 1,052 admissible pairings; the circulant structure gives examples of how folding loses weight (SI Section S3.1). Searching the same space harder would not help, so we explored three other design spaces (Table 2, Fig. 2a).

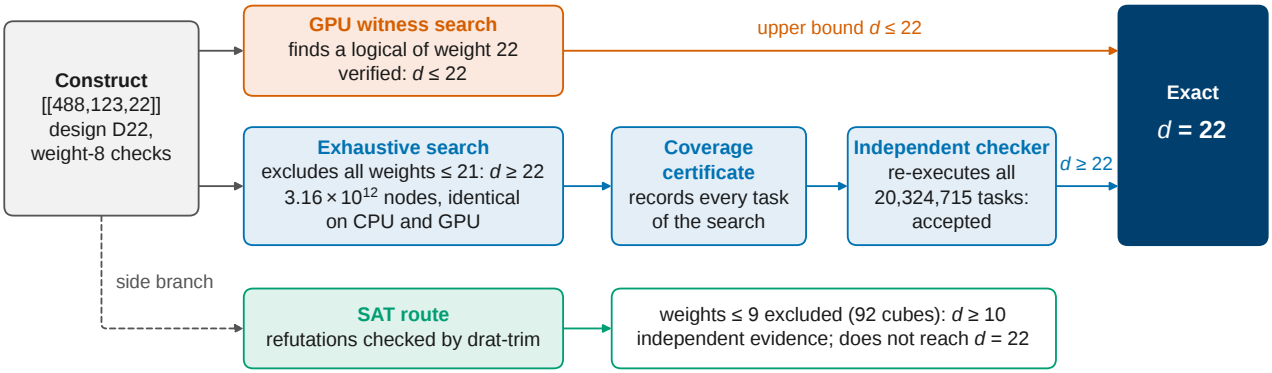


Fig. 1. How HyQEC establishes an exact distance, with [[488,123,22]] as the example. The upper bound comes from a verified logical, the lower bound from an exhaustive exclusion, and the distance is exact when the two meet. The diagram is schematic; other results rest on the evidence listed in Table 1.

Table 1. Evidence for lower bounds, and the results that rest on it. Upper bounds always come from verified witnesses. The categories are not exclusive: [[488,123,22]], for example, rests on both a re-executed certificate and two executors. The checker re-executes the same proof method; it is a second implementation, not a second mathematical proof. Of the 53 board entries established by branch and bound alone, the board's [[452,115,18]] and [[336,10,28]] have independently checked inputs and appear in stronger evidence categories, leaving 51 in the last row. For the remaining 51 entries, the span and logical-basis conditions were not checked independently of the preparation (Section 5.3).

Lower-bound evidence	Checkable outside the run	Results resting on it
Coverage certificate re-executed in full by a separately written checker	Yes, by re-execution (checker released; the [[452,115,18]] certificates regenerable from public files, others on request)	[[488,123,22]]; the board's weight-8 leader [[452,115,18]]; a plain-fold [[452,115,18]] instance from Lee et al.'s released data (o04829)[17]
SAT refutation checked by drat-trim	Yes; the encoding is audited software	125 board entries, up to $d = 13$; one degree-5 code
Two executors (CPU, GPU) with identical node totals	No; internal cross-check	[[488,123,22]], [[568,143,22]]
Independent exhaustive enumeration	No; internal cross-check	The degree-5 codes
Single complete execution with audited coverage records	No	[[336,10,28]]; [[485,97,25]], with additional pruning rules
Recorded branch-and-bound run	No	The remaining 51 board entries certified by branch and bound alone

Table 2. Three construction routes. They optimize different targets and should not be ranked by a single score. Commuting checks at every size do not imply a good distance at every size.

Route	Core idea	Representative result	Lower-bound evidence	What it shows
Split Pauli-CPM (design D22)	Build a non-CSS code directly; overlaps at which checks would anticommute are paired so that they cancel, and because the condition involves only cyclic shifts, one design gives commuting checks at every size	[[488,123,22]]: $kd^2/n = 122.0$, $n/k = 3.97$; also [[568,143,22]]	Certificate re-executed; two executors	48% above the board's weight-8 leader, exact against exact
Masked reverse fold	Reverse-fold a sparse parent with blocks removed	[[485,97,25]]: $kd^2/n = 125.0$, $n/k = 5.00$	Single execution with extra pruning rules	Highest score among the new codes, at a higher overhead; beyond the bound for the tested full parents
Degree-5 labels	Put every qubit in only five checks, using all three Pauli labels at each qubit	[[184,69,7]]: $kd^2/n = 18.4$, $n/k = 2.67$	Branch and bound and independent enumeration	Lowest overhead in our survey for $n \le 2000$, $w \le 8$, $d \ge 7$

Split Pauli-CPM. The checks of a quantum code must commute, and two Pauli checks commute exactly when they anticommute on an even number of qubits. In a Split Pauli-CPM code, six blocks of weight-8 checks act on eight blocks of qubits arranged in cycles of length P, and every qubit is acted on by X in three checks and by Z in three. Each check is a cyclic shift of a template, so where two checks overlap is fixed by the difference of their shifts. The design pairs every overlap at which two checks would anticommute with a second such overlap at the same relative shift, so that the two contributions cancel (SI Lemma S1). Because this condition involves only relative shifts, one design gives commuting checks at every cycle length P, though not necessarily a good distance at every P. A search over 1,690 admissible label patterns, which also rejected designs whose structure forces light logicals at every size, produced design D22, selected for its high score after screening out structurally weak designs. It gives [[488,123,22]] at $P = 61$ and [[568,143,22]] at $P = 71$, both with exact distance 22; at other sizes we have upper bounds only (SI Section S3.3). Because the D22 checks contain no Y, each code has a CSS counterpart, its symplectic double, with the same rate, check weight and exact distance (SI Proposition S1). The gain of D22 therefore belongs to this construction rather than to non-CSS codes as such. The verified minimum-weight logicals contain no Y, so the CSS doubles are exactly [[976,246,22]] and [[1136,286,22]].

Masked reverse fold. [[485,97,25]] comes from a fold of a different kind. Its parent is a sparse 4×10 circulant code with two blocks removed from each row of checks, and it is folded by reversal at $P = 97$, which applies the reverse-fold mechanism of Lee et al.[17] to a masked parent. This fold exceeds the bound established for the tested full 3×8 parents, not a general theorem about full-parent folds; we have not established why the masked parent escapes that tested family's bound. Its figure of merit, 125.0, is the highest we found, but its overhead, $n/k = 5.0$, is also higher than that of the D22 codes, and its distance rests on a single complete execution with additional pruning rules. A second split design, D20, gives exact distance 20 at $P = 53$, 59 and 113 (SI Section S3.5).

Degree-5 codes. The third route asks in how few checks each qubit can take part, since fewer checks per qubit can simplify the required interactions. An error on a qubit is detected by every check that acts on that qubit with a different Pauli label. With five checks per qubit, the most balanced CSS split has three X labels and two Z labels, so an X error is seen by only two checks (Fig. 2a). For CSS codes, Hsieh, Li and Lin prove $kd = O(n \log n)$ when every qubit belongs to at most two checks of at least one Pauli type, that is, $\min\{\deg_X(q), \deg_Z(q)\} \leq 2$ for every q[26]. Thus every CSS code with at most five checks per qubit has $d = O(\log n)$ at constant rate. With the non-CSS labels X, X, Y, Z, Z, every single-qubit error is seen by at least three checks, so this pattern avoids the hypothesis of their theorem (SI Section S3.6). At degree five, mixing Pauli types is necessary to avoid that hypothesis, although doing so does not by itself guarantee a large distance. Our degree-5 codes, such as [[184,69,7]] and [[248,93,7]], reach $n/k = 8/3$, the lowest among the surveyed records with $n \leq 2000$, $w \leq 8$ and $d \geq 7$, and their distances are confirmed by branch and bound and by an independent exhaustive enumeration. For the successful paired template, an explicit logical operator proves distance at most seven at every lift size (SI Section S3.6); this bound does not apply to degree-5 codes in general.

Scope of the comparisons. The 48% is relative to the board's leader in the same category, with both distances exact (Fig. 2b). Against the best previously published code in our survey with $n \leq 1000$ and $w \leq 8$, the CSS code [[776,198,20]] with $kd^2/n = 102.1$[17], whose distance is a reported value we did not certify, [[488,123,22]] is 19.5% higher, in a comparison across code types. Our survey of 5,684 records ranks only codes with certified or reported lower bounds; it omits large catalogues and is not a census (SI Section S3.7), and the board's CSS code [[928,238, $\leq$ 22]] has an upper-bound score of 124.1. A higher figure of merit does not imply a lower overhead, and neither figure says how a code performs in a fault-tolerant machine, with decoding, circuit noise and logical gates, which this report does not evaluate. Full parameters are in SI Table S4.

4 Revisiting a public leaderboard

We applied HyQEC to the non-CSS board in two steps: a search for lighter logicals on the 184 entries of a first snapshot (16 October 2026), then exact certification of the 187 entries of a second (18 October)[10]. Fig. 3a summarizes the outcome: 178 distances established exactly, three claims refuted and six entries still unresolved.

Refuted claims. Three claimed distances were too high. Generic search found lighter logicals for [[155,6,23]] and [[155,6,21]], of weights 21 and 20. For the top entry, [[662,122,95]], the lightest logical found in two generic-search passes had weight 151. Exploiting its origin as the image of a cyclic CSS code under two-qubit Cliffords, a search restricted to the corresponding subspace found a logical of weight 86 (Fig. 3b). This lowers the upper bound from 95 to 86 but does not settle the distance, whose proven lower bound is only 10. Structured searches thus catch errors that generic ones miss, and a refutation is not an exact value. Four CSS parents of non-CSS folds are overstated too (SI Section S5). The other 181 entries of the first snapshot were reached again at their claimed weight, which is evidence short of proof.

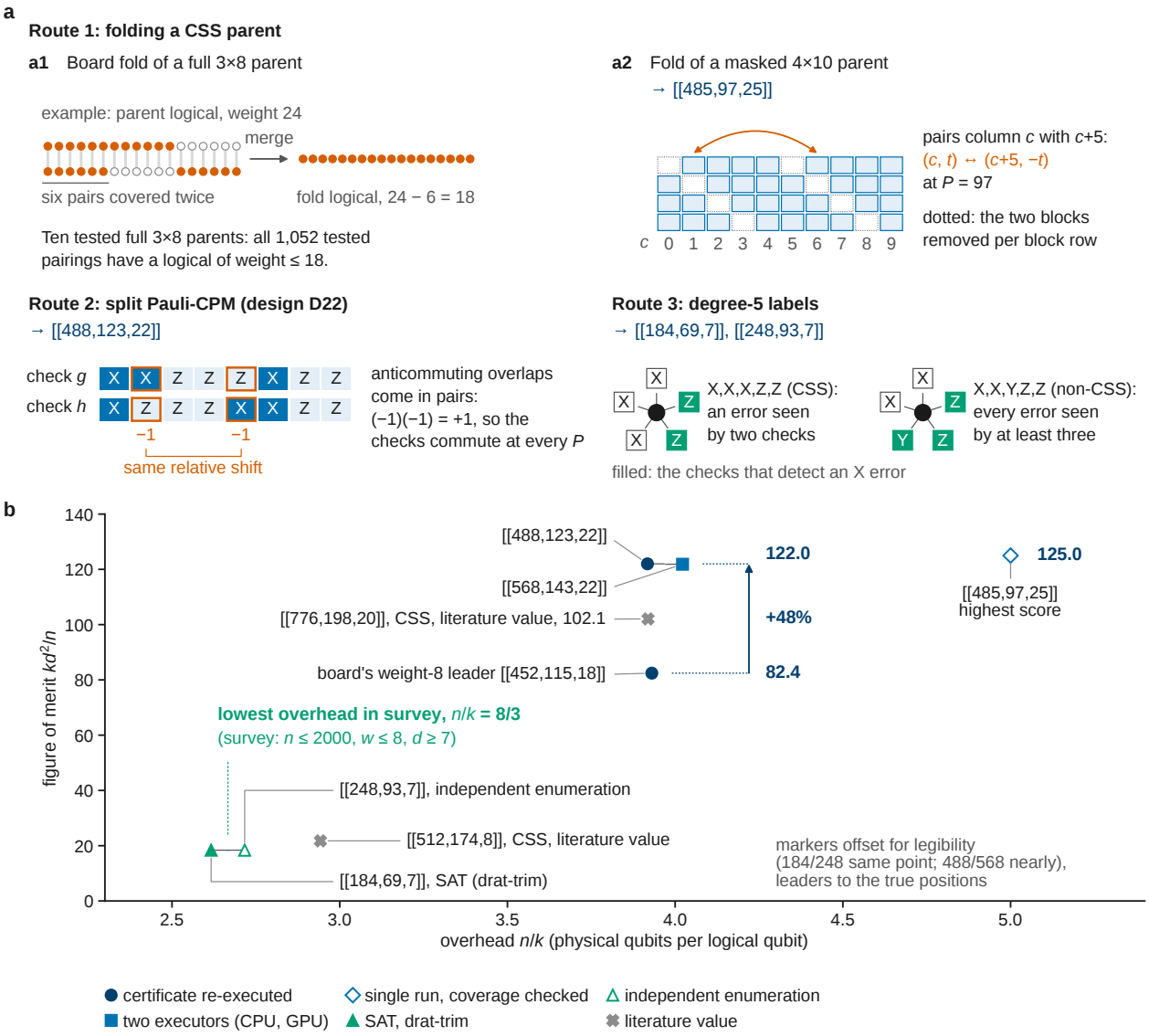


Fig. 2. New codes. **a**, The ideas behind the three routes, schematic. **b**, Results and trade-offs: the figure of merit rises from 82.4 for the board's weight-8 leader to 122.0 for [[488,123,22]] and 125.0 for [[485,97,25]], while the degree-5 codes reach the lowest overhead within the survey ($n \leq 2000$, $w \leq 8$, $d \geq 7$), without a claim of a global minimum. Markers show the evidence behind each lower bound.

Certified distances. Exhaustive proofs then established 178 distances exactly, all equal to the claimed values and ranging from 3 to 28; 125 rest on both branch and bound and SAT, and 53 on branch and bound alone (Table 1; per-entry status in SI Fig. S5). All 178 are genuinely non-CSS: none is equivalent to a CSS code under local Clifford operations (SI Section S5.2). To our knowledge, the SAT certificates for 124 of these entries are the first checkable distance certificates for non-CSS codes larger than the five-qubit code, whose Lean proof[20], however, rests on a stronger trust base than a DRAT refutation of an audited encoding. By branch and bound, the largest computationally established non-CSS distance in our survey is now 28, for [[336,10,28]] in a single complete execution, against 20 in the literature we surveyed, and the board's leaders at check weights 6 and 8 are settled. The largest searches visit 10^{12}–10^{14} nodes and take tens of hours (SI Table S5).

Unresolved entries. The six unresolved entries (SI Table S6) show where exact certification stops: four have check weight 24–32, and the other two have large claimed distances, 19 at $n = 200$ and 39 at $n = 761$. These conclusions hold only if the tools are correct, which Section 5 examines.

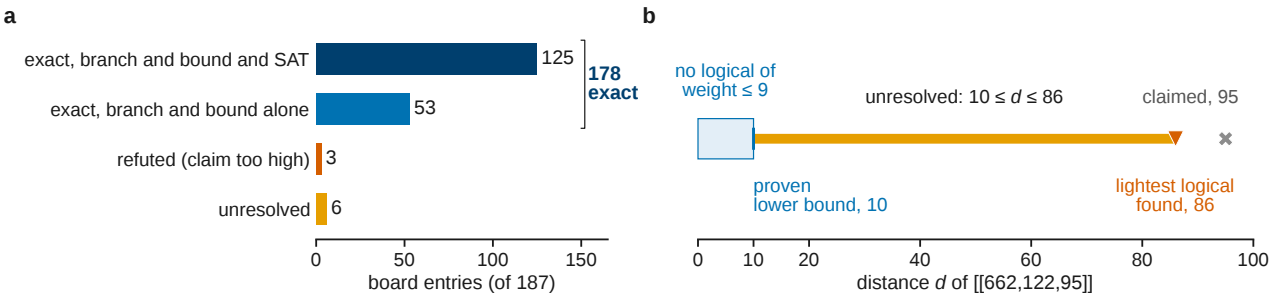


Fig. 3. Revisiting the non-CSS leaderboard. **a**, Outcome of the audit; the status of every entry is in SI Fig. S5. **b**, A refuted claim is not an exact value: the top entry's upper bound falls from 95 to 86, but its distance is still open.

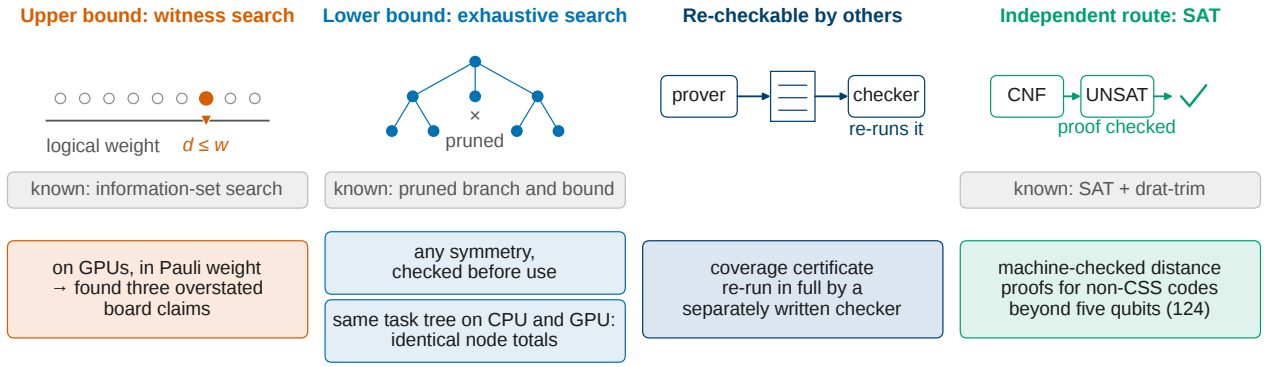


Fig. 4. What is known and what HyQEC adds. Grey: established ingredients, with references (information-set search[13–15]; pruned branch and bound[16,17]; SAT with drat-trim[24]); colour: this work's additions and what each enables.

5 Verification and reliability

5.1 How the proofs work, and what is new

The exhaustive search builds candidate errors qubit by qubit and must rule out every one lighter than the target. The key is to discard whole families of candidates without examining them one by one. A partially built error violates some checks, and each violated check needs at least one more error on its remaining qubits before the candidate can become a logical. Checks whose remaining qubits do not overlap need separate errors, so their number is a lower bound on the errors still to come. If that bound exceeds the remaining budget, every completion of the partial error is too heavy, and the whole branch is discarded. This packing bound and the branching rule are given in Lee et al.'s Appendix B.2[17]; the search also builds on the earlier irreducible-cluster approach of Dumer, Kovalev and Pryadko[16]. The bound and branching rule are proved never to discard a logical within the budget (SI Theorem S1). On the codes of this report, switching the bound off multiplies the number of explored nodes 1,300- to 5,200-fold.

HyQEC's contribution lies in what surrounds this principle. Its witness search treats every qubit as a unit, so that logicals containing Y are found as readily as others. Its symmetry reduction verifies every symmetry before using it. Its deterministic decomposition into replayable tasks lets CPU and GPU executors explore the same tree with identical node counts and lets a separately written checker re-execute the whole search. And an independent SAT method, with refutations checked by drat-trim, confirms the results where it is feasible. Together, these turn a search that would otherwise have to be trusted into evidence that others can check.

5.2 Where the proofs scale, and where they do not

On one GPU the witness search reaches 8–55 times the information-set rate of the board's 8-thread CPU engine, a comparison across hardware (eight benchmark codes; SI Fig. S3). Against dist-m4ri, whose connected-cluster method is one of codeDistance's exact backends[18], run at the same time on equal sets of 64 cores, our prover settled [[452,115,18]] in 1,234 s, while dist-m4ri had reached weight 13 after two hours. On small dense codes the balance reverses: dist-m4ri was faster in 15 of 21 comparisons when preparation was excluded. Separately, codeDistance's Brouwer–Zimmermann method was faster on five of eight small codes using full process wall time for both tools. The SAT method certifies exact distances only up to $d = 13$ and reaches only $d \geq 10$ on the two distance-22 codes and $d \geq 9$ on the distance-25 code, so branch and bound carries the large cases (SI Section S5.5).

5.3 What was checked, and what was not

The exhaustive search is sound only if its inputs are right: the checks must span the code, the logical basis must be complete, and every symmetry must preserve the code. The SAT encoder asserts the first two conditions; the branch-and-bound preparation computes them without asserting them. They were checked independently for the certificate instances, the two distance-22 codes, [[336,10,28]] and [[485,97,25]], and through agreement with SAT for 125 entries; for 51 of the 53 board entries certified by branch and bound alone, those conditions have not been checked beyond the preparation; the other two have independent input checks. The recorded cross-reviews found nine defects through which a tool could have produced a false certificate, in four components: the checkpoints of the research tool, a pruning rule, a rejected draft and a verification layer of the release package (SI Section S6; Section 6 describes how one was found). An audit of 580 historical result records found no reported result affected by the two checkpoint defects.

Three large full-distance certificates have been re-executed in full, alongside smaller conformance cases. The checkpoint fixes have since received independent review and crash testing. The remaining limits are that [[336,10,28]] and [[485,97,25]] rest on single executions, none of our distance proofs is formalized in a proof kernel, and outside re-checking awaits the artifacts.

6 Discussion

Competitive code parameters need certified distances. The figure of merit grows with d^2, the most overstated claim on the non-CSS board was its top entry, and even our literature baseline carries a reported rather than a certified distance. Comparisons are meaningful exact against exact; otherwise they should state the status of both sides.

Exact distances of large structured codes are within reach, at a cost. Sparsity, symmetry and pruning make exact certification feasible for several important instances with hundreds of qubits and distances above 20. But the open board entries sit at high check weight or large distance, existing tools remain faster on small dense codes, and only three large full-distance certificates have been re-executed in full. None of this extends to random codes or to arbitrary large stabilizer codes.

Leaderboards should separate found upper bounds from certified distances. On the non-CSS board every value was an upper bound; 178 proved exact, so most claims were right, but without certification no one could tell which, and the claimed bound of 95 remained at least 10% too high although the lightest logical found in our generic-search passes had weight 151. At the cited snapshot, the board already recorded re-derivable DRAT-checked proof logs for CSS codes: their CNF formulas can be regenerated, refuted with Kissat and checked by drat-trim[10]. Extending this mechanism to non-CSS codes, and adding support for coverage certificates, would let it distinguish their distance-evidence levels. Our certificates have not yet been submitted.

The role of non-CSS structure is specific, not general. At weight 8, the CSS doubles of D22 have the same rate, check weight and exact distance, so its gain belongs to the construction. At qubit degree five, mixed Pauli labels are necessary to avoid the hypothesis of the CSS bound of Ref.[26]; the $(2, 2, 1)$ pattern achieves this, without establishing superlogarithmic distance growth. And the fold obstruction is established for one tested class of parents. Searches for non-CSS codes are most promising where CSS structure provably falls short.

Better parameters are not yet better machines. A higher figure of merit is a statement about code parameters. Whether these codes lower the overhead of a working fault-tolerant machine depends on decoding, syndrome-extraction circuits, connectivity and logical gates, none of which is evaluated in this report.

The role of Hyra 2.0. HyQEC was developed by Hyra 2.0, an AI-agent research system built by the Hyra Team at Tencent Hunyuan, in which an Overall agent talks to the user, mission leads own research problems, and workers explore, build, run and review (SI Section S7). A member of the team (M.G.) set the goals, the scope and the acceptance criteria, including the three requirements for the verifier: proof time reported apart from search time, generality to non-CSS codes, and evidence that others can check. Within them, the agents designed and implemented the new code, ran the experiments and chose which claims to audit. One episode shows how the system's checks worked in practice. The agent building the release package flagged that the original prover's checkpoints recorded which code was being searched but not the search bound. An agent of the other model family, acting as referee, turned this into a concrete false certificate: a finished run resumed with a larger bound reported $d \geq 5$ for the five-qubit code, whose distance is 3. Over roughly two hours after that reproduction the builder fixed the release package, another agent found a second gap in the same mechanism, a fact-checking agent audited 580 historical result records and found none affected by those checkpoint defects, and a reviewer rejected a first redesign with counterexamples. In the frozen correction record, agents made every first detection of a numerical or soundness error, but without a control the record cannot show that this organization

outperforms simpler ones. Related systems carry much of the research cycle in machine learning[27,28], chemistry[29] and biomedicine[30,31], produce checked constructions and proofs in mathematics[32–34], and build verification and cross-model audits into their workflow[35–37].

Next steps. Formalization of the proofs; submission of the certificates to the board; release of the artifacts; certification of the literature baseline [[776,198,20]]; evaluation of decoding; and a controlled comparison of research organizations on a fixed set of tasks with a common evidence standard and a matched budget.

Code availability. The certifier hyqec-verify 0.3.0 and the independently reviewed checker hyqec-check 2.0.2, licensed under Apache-2.0, are available at `https://github.com/Tencent-Hunyuan/hyra-results/tree/main/AI4Science/hyqec`. The package includes the GPU witness search and SAT pipeline. The construction code and historical research-tool versions are available from the author on request. The release package's README reports a separate, budget-limited sweep with a pre-release build of hyqec-verify on the first snapshot: 165 of its 184 entries proven exact within 300 s on CPU or 30 min on one GPU per code. The 178 exact distances reported here come from longer runs of the research tools on the second snapshot.

Data availability. Certificates, witnesses and run records are available from the author on request, together with the input matrices, checkpoints, ledgers and survey records needed to trace the reported results. The board snapshots are commits 2f1f65ea and fa4b0de3 of the public challenge repository[10].

Supplementary Information. The SI gives formal definitions, the design decisions and trust boundary, the constructions with their proofs, the algorithms and the soundness proof, the full evaluation, the known defects, the development record, the artifacts and instructions for re-checking a certificate.

Acknowledgements. The author thanks the maintainers and contributors of the Unitary Foundation qLDPC challenge for the public board and its verifier.

Author contributions. M.G. (Hyra Team) conceived and directed the project, set its goals, scope and acceptance criteria, and wrote the manuscript. Hyra 2.0 designed and implemented the software, ran the experiments and analyses, and prepared figures and supporting text under M.G.'s direction.

Competing interests. The author declares no competing interests.

References

[1] Dennis, E., Kitaev, A., Landahl, A. & Preskill, J. Topological quantum memory. *J. Math. Phys.* **43**, 4452–4505 (2002).

[2] Fowler, A. G., Mariantoni, M., Martinis, J. M. & Cleland, A. N. Surface codes: towards practical large-scale quantum computation. *Phys. Rev. A* **86**, 032324 (2012).

[3] Gidney, C. & Ekerå, M. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum* **5**, 433 (2021).

[4] Gidney, C. How to factor 2048 bit RSA integers with less than a million noisy qubits (2025). ArXiv:2505.15917.

[5] Gottesman, D. Fault-tolerant quantum computation with constant overhead. *Quantum Inf. Comput.* **14**, 1338–1372 (2014); arXiv:1310.2984.

[6] Panteleev, P. & Kalachev, G. Asymptotically good quantum and locally testable classical LDPC codes. *Proc. 54th STOC*, 375–388 (2022); arXiv:2111.03654.

[7] Leverrier, A. & Zémor, G. Quantum Tanner codes. In *Proc. 63rd IEEE FOCS*, 872–883 (2022). `https://doi.org/10.1109/FOCS54457.2022.00117`; arXiv:2202.13641.

[8] Breuckmann, N. P. & Eberhardt, J. N. Quantum low-density parity-check codes. *PRX Quantum* **2**, 040101 (2021); arXiv:2103.06309.

[9] Bravyi, S. *et al.* High-threshold and low-overhead fault-tolerant quantum memory. *Nature* **627**, 778–782 (2024); arXiv:2308.07915.

[10] Unitary Foundation. QEC Challenge: a public, automatically verified leaderboard for quantum LDPC codes. `https://unitaryfoundation.github.io/qldpc-challenge/` (2026). Snapshots: commits 2f1f65ea (16 October 2026) and fa4b0de3 (18 October 2026).

[11] Vardy, A. The intractability of computing the minimum distance of a code. *IEEE Trans. Inf. Theory* **43**, 1757–1766 (1997).

[12] Kapshikar, U. & Kundu, S. On the hardness of the minimum distance problem of quantum codes. *IEEE Trans. Inf. Theory* **69**(10), 6293–6302 (2023). `https://doi.org/10.1109/TIT.2023.3286870`; arXiv:2203.04262.

[13] Prange, E. The use of information sets in decoding cyclic codes. *IRE Trans. Inf. Theory* **8**, 5–9 (1962).

[14] Leon, J. S. A probabilistic algorithm for computing minimum weights of large error-correcting codes. *IEEE Trans. Inf. Theory* **34**, 1354–1359 (1988).

[15] Pryadko, L. P., Shabashov, V. A. & Kozin, V. K. QDistRnd: a GAP package for computing the distance of quantum error-correcting codes. *J. Open Source Softw.* **7**, 4120 (2022).

[16] Dumer, I., Kovalev, A. A. & Pryadko, L. P. Distance verification for classical and quantum LDPC codes. *IEEE Trans. Inf. Theory* **63**, 4675–4686 (2017); arXiv:1611.07164.

[17] Lee, J. Y., Okada, K., Maskara, N., Kasai, K. & Zhou, H. Design principles for ultra-high-rate quantum codes (2026). ArXiv:2609.30069.

[18] Webster, M., Jacob, A. & Higgott, O. Distance-finding algorithms for quantum codes and circuits (2026). ArXiv:2603.22532.

[19] Ehatamm, M., Lee, Y., Wu, X. & Tao, R. End-to-end formalization of quantum error correction (2026). ArXiv:2605.16523.

[20] An, S. & Yang, F. Quantum error-correcting code parameters, checkable by a certificate of provable size (2026). ArXiv:2610.03214.

[21] Liu, Z. & Marquardt, F. Large-language-model discovery of quan-

tum LDPC codes through structured concept evolution (2026). ArXiv:2606.24808.

[22] Yan, G. *et al.* OmniQEC: discovering practical quantum error-correcting codes by an AI scientist (2026). ArXiv:2607.25865.

[23] He, X., Lu, S. & Zeng, B. Co-designing quantum codes with transversal diagonal gates via multi-agent systems. In *Proc. 2026 IEEE 19th Dallas Circuits and Systems Conference (DCAS)*, 1–6 (2026). https://doi.org/10.1109/DCAS69364.2026.11544392. See also arXiv:2510.20728v3 (2026) for the Lean verification results.

[24] Wetzler, N., Heule, M. J. H. & Hunt, W. A., Jr. DRAT-trim: efficient checking and trimming using expressive clausal proofs. In *SAT 2014*, LNCS 8561, 422–429 (Springer, 2014).

[25] Okada, K. & Kasai, K. Pair-partition constructions for CPM-based quantum LDPC codes (2026). ArXiv:2607.14091.

[26] Hsieh, M.-H., Li, X. & Lin, T.-C. Simplified quantum weight reduction with optimal bounds. arXiv:2510.09601v2 (2026; first posted 2025).

[27] Lu, C. *et al.* The AI Scientist: towards fully automated open-ended scientific discovery (2024). ArXiv:2408.06292.

[28] Yamada, Y. *et al.* The AI Scientist-v2: workshop-level automated scientific discovery via agentic tree search (2025). ArXiv:2504.08066.

[29] Boiko, D. A., MacKnight, R., Kline, B. & Gomes, G. Autonomous chemical research with large language models. *Nature* **624**, 570–578 (2023).

[30] Swanson, K., Wu, W., Bulaong, N. L., Pak, J. E. & Zou, J. The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. *Nature* **646**, 716–723 (2025).

[31] Gottweis, J. *et al.* Accelerating scientific discovery with Co-Scientist. *Nature* **655**, 487–496 (2026).

[32] Romera-Paredes, B. *et al.* Mathematical discoveries from program search with large language models. *Nature* **625**, 468–475 (2024).

[33] Novikov, A. *et al.* AlphaEvolve: a coding agent for scientific and algorithmic discovery (2025). ArXiv:2506.13131.

[34] Hubert, T. *et al.* Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature* **651**, 607–613 (2026).

[35] Deng, K. *et al.* Towards verifiable and self-correcting AI physicists for quantum many-body simulations (2026). ArXiv:2604.00149.

[36] Kon, P. T. J. *et al.* Curie: toward rigorous and automated scientific experimentation with AI agents (2025). ArXiv:2502.16069.

[37] Dong, Z. & Chen, Y. CrossAudit: a Git-native, cross-vendor audit loop for agentic science (2026). ArXiv:2608.28631.